

Indexación y Estrategias de Chunking

Módulo 7 · Sistemas RAG y Gestión del Conocimiento

Fixed-size, semantic, parent-child y Long RAG: como la estrategia de chunking determina la calidad de recuperación.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Indexación y Estrategias de Chunking

El chunking es el proceso de dividir documentos en fragmentos más pequeños para indexación y recuperación. Es uno de los parámetros más determinantes de la calidad de un sistema RAG, y también uno de los más subestimados. Un chunking mal diseñado produce un sistema que recupera fragmentos irrelevantes, que corta el contexto en puntos que eliminan el significado, o que produce chunks demasiado pequeños para responder preguntas que requieren información de múltiples secciones.

Los sistemas RAG de 2024-2025 han evolucionado significativamente en estrategias de chunking. El chunking de tamaño fijo (fixed-size) fue el primero y sigue siendo el más simple de implementar. El semantic chunking mejora el recall hasta un 9% sobre el chunking fijo según benchmarks de 2025. El Long RAG (chunks más grandes) mejora la eficiencia un 30-40% en sistemas con contextos largos. La elección correcta depende del tipo de documentos, las consultas esperadas, y las restricciones de latencia y coste.

Las estrategias de chunking

Fixed-size chunking: simple pero limitado

El chunking de tamaño fijo divide el documento en ventanas de N tokens con un solapamiento (overlap) de M tokens entre chunks consecutivos. Por ejemplo, chunks de 512 tokens con 50 tokens de overlap. Simple de implementar, consistente, y fácil de razonar sobre el coste de embedding. La desventaja es que los cortes son arbitrarios: pueden partir una oración a la mitad, separar una pregunta de su respuesta, o cortar en medio de una definición. El overlap mitiga parcialmente este problema (la información al borde de un chunk aparece también en el chunk siguiente).

Semantic chunking: cortar en fronteras naturales

El semantic chunking identifica fronteras naturales en el texto para cortar: párrafos, secciones (identificadas por headings), cambios de tema (detectados por análisis de similitud entre oraciones consecutivas). Para documentos estructurados con markdown, HTML, o PDF con headings reconocibles, el corte por headings produce chunks semánticamente coherentes y relevantes. Langchain y LlamaIndex tienen implementaciones de semantic chunking que detectan cambios de tema usando el embedding de oraciones consecutivas.

Parent-child chunking: recuperar detalle con contexto

Parent-child chunking (o multi-granularity chunking) mantiene dos niveles de chunks: los "hijos" son chunks pequeños (200-500 tokens) para búsqueda precisa, y los "padres" son chunks mayores (500-2000 tokens) o documentos completos que se usan como contexto en el prompt de generación. Cuando se recupera un chunk hijo, se incluye en el prompt su chunk padre para proporcionar el contexto completo. Esto combina la precisión de la búsqueda con el contexto necesario para la generación.

Long RAG: chunks grandes para modelos con contexto largo

Con modelos que tienen ventanas de contexto de 128K+ tokens (GPT-4, Claude), Long RAG procesa chunks más grandes (secciones o documentos enteros). En lugar de recuperar K chunks pequeños, recupera 2-5 secciones grandes. Esto reduce la fragmentación del contexto, mejora la coherencia de las respuestas, y reduce el número de registros en la base vectorial. El trade-off es mayor coste por consulta (más tokens al LLM) y potencialmente mayor latencia.

SECCIÓN 3 · CÓMO FUNCIONA

Implementación práctica de las estrategias de chunking

LangChain proporciona implementaciones de las principales estrategias: RecursiveCharacterTextSplitter para fixed-size (con separadores recursivos: primero intenta dividir por párrafos, luego por oraciones, luego por palabras), SemanticChunker para semantic chunking basado en embeddings, y MarkdownHeaderTextSplitter para documentos markdown. LlamaIndex tiene SimpleNodeParser, SentenceSplitter, y SemanticSplitterNodeParser.

Los metadatos son tan importantes como el contenido del chunk. Cada chunk debe incluir: source (documento de origen), page o section (posición en el documento), title (título del documento o sección), date (fecha del documento), y cualquier metadato de negocio relevante (categoría, departamento, nivel de confidencialidad). Los metadatos permiten filtrar la búsqueda y generar citas precisas.

Preprocesamiento específico por tipo de documento

Cada tipo de documento requiere un pipeline de preprocesamiento distinto antes del chunking. PDF: usar pdfminer o pypdfium2 para extracción de texto, docling para documentos complejos con tablas e imágenes. Word/DOCX: python-docx con extracción de headings para chunking estructural. HTML: beautiful-soup o markdownify para convertir a markdown antes del chunking. Código: language-specific parsers (tree-sitter) para dividir por función, clase o módulo. Para PDFs escaneados: OCR con pytesseract o AWS Textract.

SECCIÓN 4 · EJEMPLOS REALES

- Chunking de contratos legales: los contratos tienen estructura de cláusulas numeradas con jerarquía (1. → 1.1. → 1.1.1.). El chunking correcto respeta esa estructura: cada cláusula (o subcláusula) es un chunk independiente con su número como metadato. Chunks de ~400-800 tokens por cláusula. El solapamiento se hace a nivel de cláusula hermana, no a nivel de caracteres.
- Chunking de documentación técnica de software: las páginas de documentación tienen secciones claramente delimitadas por headings (H1, H2, H3). El chunking por heading produce chunks semánticamente coherentes que corresponden a funciones o conceptos específicos. Los ejemplos de código se mantienen intactos dentro del chunk para no romper el contexto.
- Chunking de emails y conversaciones: las conversaciones tienen turnos con contexto dependiente entre sí. Fixed-size chunking rompe el contexto conversacional. El chunking correcto agrupa por conversación completa (si cabe) o por bloque de N turnos consecutivos con overlap de 1-2 turnos.

- Chunking de papers científicos: abstract, introducción, metodología, resultados, y discusión son secciones semánticamente distintas. El chunking por sección (identificada por el heading o por estructura de PDF) produce chunks que responden a diferentes tipos de preguntas sobre el paper.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: El mismo tamaño de chunk para todos los tipos de documentos. Un tamaño de chunk de 512 tokens puede ser demasiado grande para preguntas sobre datos de una tabla (donde 50-100 tokens por fila son suficientes) y demasiado pequeño para preguntas sobre procedimientos complejos (donde 1000-2000 tokens son necesarios). Adapta el tamaño a cada tipo de documento.

Error 2: Olvidar el overlap entre chunks. Sin overlap, las preguntas cuya respuesta está al borde de dos chunks consecutivos no serán respondidas correctamente. El overlap de 10-20% del tamaño del chunk es una práctica estándar. En semantic chunking, el overlap se logra incluyendo la última oración del chunk anterior al principio del siguiente.

Error 3: No limpiar el texto antes del chunking. PDFs con columnas múltiples, encabezados de página, notas al pie, y artefactos de conversión contaminan los chunks con texto irrelevante que degrada la calidad del embedding. La limpieza del texto (eliminar encabezados/pies de página repetitivos, corregir el orden de columnas, eliminar artefactos) antes del chunking es un paso que impacta significativamente la calidad final.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El proceso de optimización del chunking para tu corpus: comienza con fixed-size chunking de 512 tokens con 50 de overlap como baseline. Evalúa el recall@5 en tu golden dataset. Experimenta con tamaños distintos (256, 512, 1024) y mide el impacto en recall. Prueba semantic chunking y mide si mejora sobre el baseline. Identifica los casos donde la recuperación falla: ¿el problema es el tamaño del chunk (demasiado pequeño → falta contexto, demasiado grande → ruido), la calidad del texto (necesita limpieza), o el modelo de embedding?

Para un pipeline de ingesta robusto en producción: define el pipeline como código (Python scripts o DAG con Airflow/Prefect). Incluye detección de cambios (si el documento no ha cambiado desde la última indexación, no re-indexar). Implementa logging de cada documento procesado (fecha, número de chunks, errores). Define una política de actualización (¿cuándo y cómo actualizar los embeddings cuando cambian los documentos fuente?).

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M7-T4 (Búsqueda semántica): los chunks son la unidad de búsqueda; la calidad del chunking determina la calidad de la recuperación.
- M7-T7 (Evaluación RAG): RAGAS mide context recall, que depende directamente de la calidad del chunking.
- M7-T6 (RAG empresarial): la elección de estrategia de chunking es una de las decisiones de arquitectura más importantes en un sistema RAG de producción.

SECCIÓN 8 · RESUMEN EJECUTIVO

El chunking divide documentos en fragmentos para indexación y recuperación. Las estrategias principales: fixed-size (simple, 256-1024 tokens con 10-20% overlap), semantic (cortar en fronteras naturales, +9% recall), parent-child (búsqueda precisa + contexto completo), Long RAG (chunks grandes para modelos de contexto largo, +30-40% eficiencia). Cada tipo de documento requiere su propia estrategia. Los metadatos (source, date, section) son tan importantes como el contenido del chunk. El proceso de optimización: baseline fixed-size → medir recall@5 en golden dataset → experimentar con variantes → elegir la mejor estrategia por tipo de documento. Limpia el texto antes del chunking; el ruido en el texto degrada la calidad del embedding.